

Assignment 6: GUI-Based Multi-Client Chat Application Using TCP

Objective

Convert the terminal-based TCP chat application developed in the previous assignment into a graphical desktop application while reusing the existing server implementation. The objective is to introduce GUI programming, event-driven programming, multithreading, and user-friendly network application development.

Prerequisite

Students must reuse and extend their Assignment 5 implementation. The server should be reused with minimal modifications. The socket communication logic should remain largely unchanged. The primary objective is to replace the terminal-based client with a graphical desktop application. Starting from scratch is not permitted.

Network Setup

Use Mininet with one server and four clients.

```
sudo mn --topo single,5
```

Topology:

h1 : Chat Server

h2 : Client A

h3 : Client B

h4 : Client C

h5 : Client D

Verify connectivity using:

nodes

net

pingall

Development Guidelines

This assignment focuses on GUI development, not networking. Reuse the TCP socket communication, server implementation, broadcast messaging, private messaging, online user management, chat history, and logging developed in Assignment 5. Build a graphical interface around the existing networking application.

Suggested Python Modules

tkinter

tkinter.ttk

tkinter.scrolledtext

threading

socket
time

Suggested GUI Layout

The interface should contain a login window with username and password (optional), followed by a chat window containing a scrollable message area, online users panel, message input box, Send button, Disconnect button, and a connection status indicator.

GUI Components (Minimum Requirement)

Use at least the following widgets:

- Labels
- Entry boxes
- Buttons
- Scrollable text area
- List of online users
- Frames
- Status label

Task 1: GUI Login Window (15 Marks)

Develop a login window containing username, password (if supported), and Connect button. Validate user input, prevent empty usernames, and display appropriate error or success messages.

Task 2: Graphical Chat Interface (25 Marks)

Replace the terminal interface with a graphical interface containing a scrollable message display, message input box, Send button, Disconnect button, and automatic scrolling.

Task 3: Online User List (10 Marks)

Display connected users and update the list automatically whenever users join or leave.

Task 4: Messaging Features (15 Marks)

Support broadcast messaging, private messaging, join notifications, and leave notifications. Private messaging may use user selection or username entry.

Task 5: Background Message Handling (15 Marks)

Receive incoming messages using a separate thread so the GUI remains responsive. Users should be able to continue typing while messages are received. Hint: Do not perform blocking socket operations in the main GUI thread.

Task 6: GUI Testing and Verification (10 Marks)

Demonstrate successful user login, broadcast messaging, private messaging, user joining, user leaving, and disconnect using multiple Mininet clients.

Task 7: Wireshark Verification (5 Marks)

Capture traffic using filter `tcp.port == 5000`. Include screenshots showing client connection, broadcast message, private message, and client disconnection with brief explanations.

Task 8: Reflection Questions (5 Marks)

Answer: Why should networking logic and GUI code be separated? Why is a background thread required? Which components from Assignment 5 were reused? How did the GUI improve usability? Suggest one future enhancement.

Required Outputs

server.py
client_gui.py
screenshots/
report.pdf

Screenshot Requirements

Include screenshots of the login window, successful connection, main chat window, broadcast messaging, private messaging, user joining, and user leaving.

GitHub Submission (Mandatory)

Starting from this assignment, each student must maintain a public GitHub repository for the remaining networking internship assignments.

The GitHub repository name must begin with: ISEA-Phase3-TezpurUniversity-

The repository should include:

- Source code
- Report
- Screenshots
- README.md

The README should contain:

- Project title
- Objective
- Software requirements
- Network topology
- Execution steps
- Sample screenshots
- Brief description of the implementation

Submit the GitHub repository link along with the ZIP file.

Report Format

1. Objective
2. GUI Design
3. System Architecture
4. Components Reused from Assignment 5
5. GUI Design Decisions
6. Testing Results
7. Wireshark Verification
8. Reflection Answers
9. Conclusion

Submission Guidelines

Submit ROLLNO_NAME_ASSIGNMENT6.zip containing:

server.py
client_gui.py
screenshots/
report.pdf

Evaluation Rubric

Component	Marks
GUI Login Window	15
GUI Chat Interface	25
Online User List	10
Messaging Features	15
Background Message Handling	15
GUI Testing	10
Wireshark Verification	5
Reflection & Report	5
Total	100

Important Notes

Reuse your Assignment 5 implementation. Keep networking logic independent of GUI code. Use a background thread for receiving messages. A simple interface is sufficient; focus on functionality rather than appearance. Test the application with multiple clients inside Mininet and be prepared to demonstrate it.